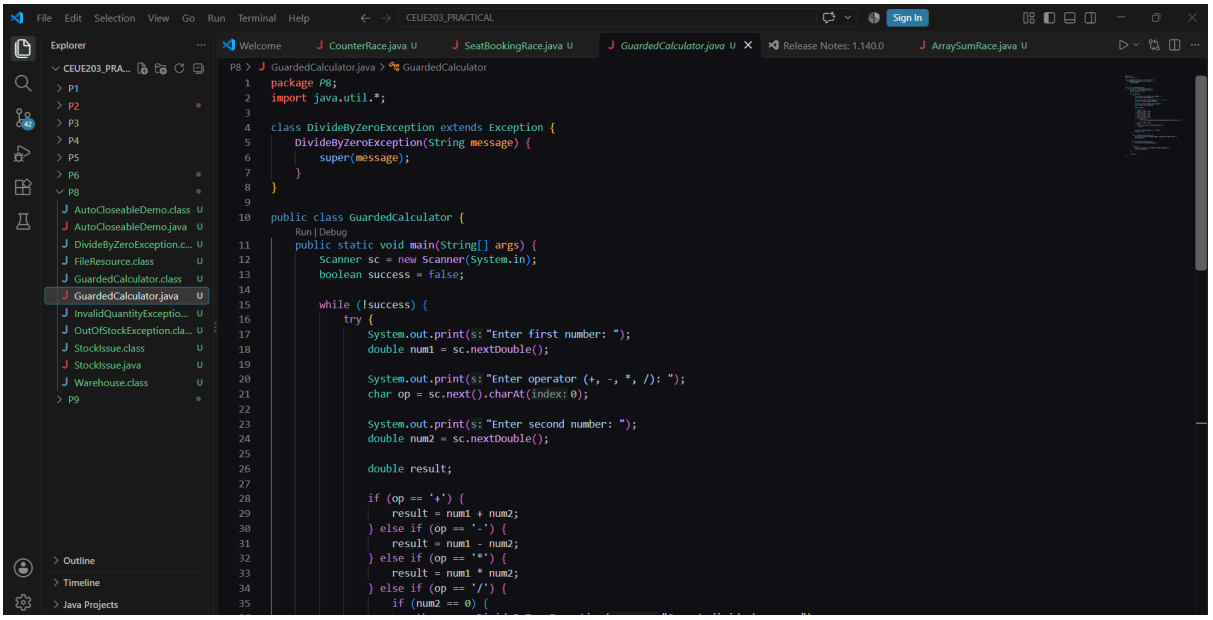
PRACTICAL 8

TASK 1 :

Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clearmessage each time; loop until a valid calculation succeeds and use finally to log each attempt.

CODE :



```java
package P8;
import java.util.*;

class DivideByZeroException extends Exception {
    DivideByZeroException(String message) {
        super(message);
    }
}

public class GuardedCalculator {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean success = false;

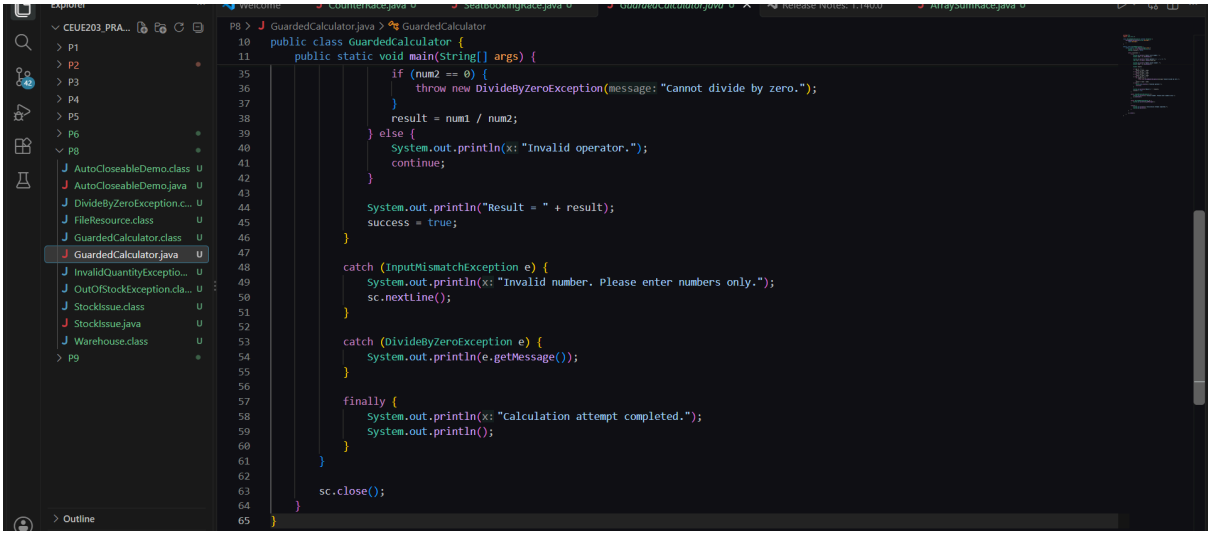
        while (!success) {
            try {
                System.out.print(s: "Enter first number: ");
                double num1 = sc.nextDouble();

                System.out.print(s: "Enter operator (+, -, *, /): ");
                char op = sc.next().charAt(index: 0);

                System.out.print(s: "Enter second number: ");
                double num2 = sc.nextDouble();

                double result;

                if (op == '+') {
                    result = num1 + num2;
                } else if (op == '-') {
                    result = num1 - num2;
                } else if (op == '*') {
                    result = num1 * num2;
                } else if (op == '/') {
                    if (num2 == 0) {
```



```java
public class GuardedCalculator {
    public static void main(String[] args) {
                    if (num2 == 0) {
                        throw new DivideByZeroException(message: "Cannot divide by zero.");
                    }
                    result = num1 / num2;
                } else {
                    System.out.println(x: "Invalid operator.");
                    continue;
                }

                System.out.println("Result = " + result);
                success = true;
            }

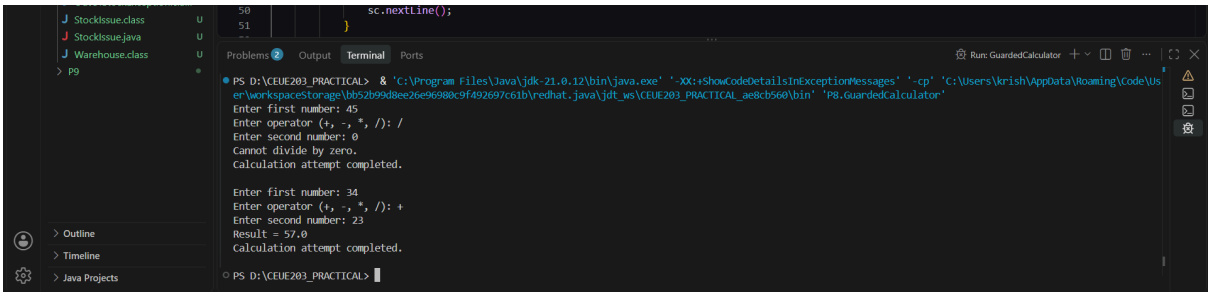
            catch (InputMismatchException e) {
                System.out.println(x: "Invalid number. Please enter numbers only.");
                sc.nextLine();
            }

            catch (DivideByZeroException e) {
                System.out.println(e.getMessage());
            }

            finally {
                System.out.println(x: "Calculation attempt completed.");
                System.out.println();
            }
        }

        sc.close();
    }
}
```

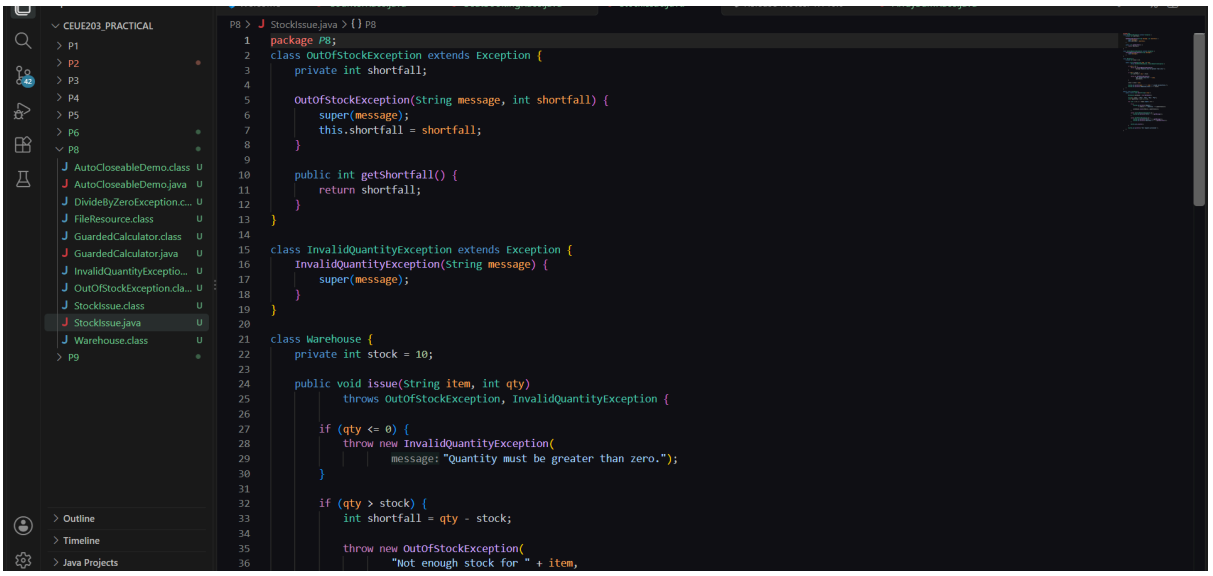
OUTPUT :

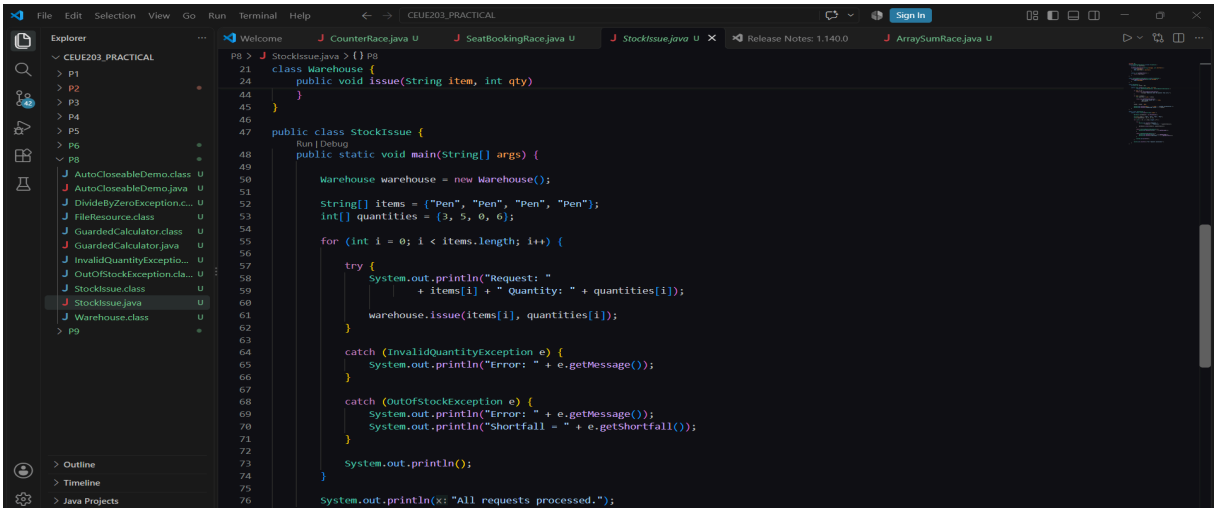


TASK 2 :

2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0.Process a list of requests, catching and reporting each failure without stopping the run.

CODE :





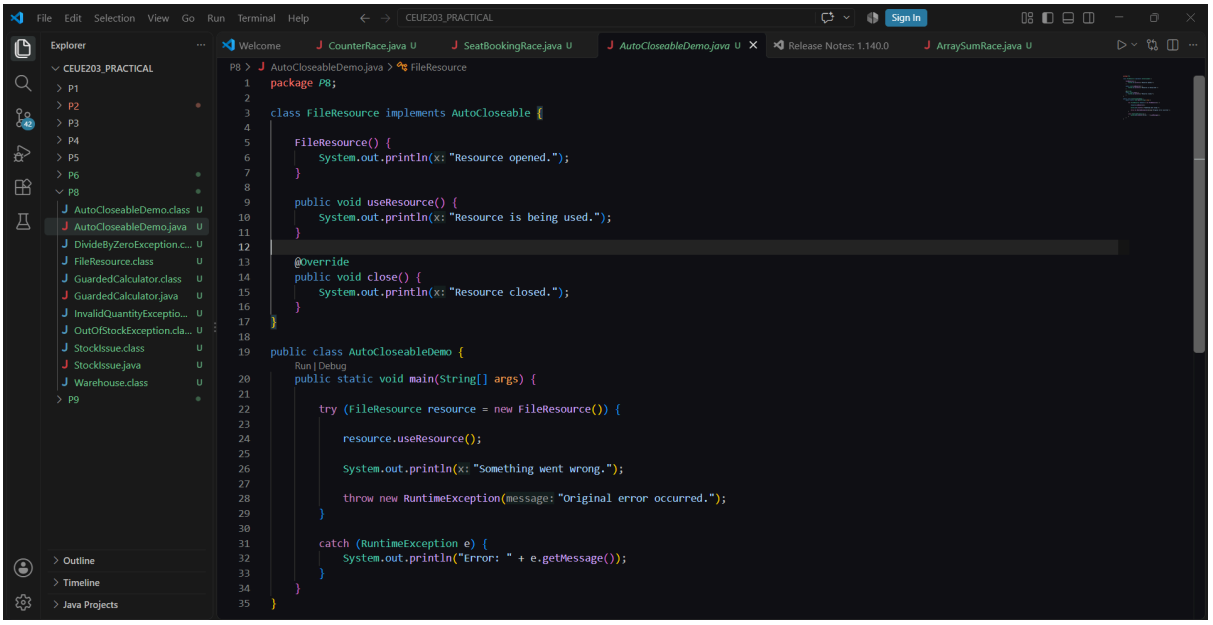
OUTPUT :



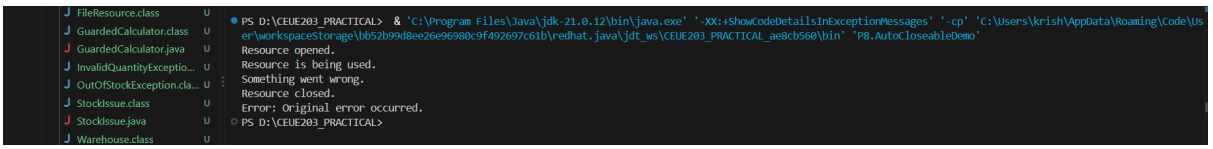
TASK 3 :

3. (For Advanced Learners – home) Create an AutoCloseable resource and use try-with-resources to prove it always closes — even when an exception is thrown inside the block and that the original error is the one reported. Decide what to model as the resource.

CODE :



OUTPUT :



KEY QUESTION :

1. **What is the difference between a checked and an unchecked exception?**
   A checked exception is checked by the compiler, so we must handle it using try-catch or declare it using throws. Example: IOException.
   An unchecked exception happens at runtime and the compiler does not force us to handle it. Example: ArithmeticException or NullPointerException.
2. **What is the purpose of the finally block, and when does it run?**
   The finally block is used to write code that should execute whether an exception occurs or not. It usually runs after the try and catch blocks and is commonly used for cleanup work.
3. **How does try-with-resources guarantee a resource is closed?**
   Try-with-resources automatically calls the close() method of the resource when the try block finishes. It closes the resource even if an exception occurs inside the block, so we do not need to close it manually.